

el tema del login perfecto, dejamos el /admin para entrar y el logo de la rueda dentada que se vea solamente al hacer login. lo del tema oscuro perfecto tambien con esos tres estados y el tema de guardar las preferencias en localStorage. para el idioma perfecto, dejamos primero en español y a posterior que no haya que duplicar pantallas con el ingles sino usar recursos de idiomas. podemos ya de primeras usar los recursos en los que de momento solo va a estar presente el español, para no tener que cambiar con el tiempo todo el formato del continente para el uso de recursos de idiomas. el logo igualmente esta bien esa idea

Mostrar más

Memoria actualizada

Perfecto. Con esto creo que **ya tenemos suficientemente definido el punto de partida** y no hace falta seguir tomando decisiones de diseño antes de comenzar. Además, lo que acabas de plantear sobre recursos de idiomas y el sistema visual es exactamente lo que haría.

Sobre el logo y los colores

No necesitamos esperar al logo definitivo para empezar.

Podemos preparar desde el principio un **sistema de diseño desacoplado del logo**, por ejemplo:

```
Design System
├─ Tipografía
├─ Colores
│   ├─ Primary
│   ├─ Secondary
│   ├─ Background
│   ├─ Surface
│   └─ Text
└─ ...
├─ Espaciados
└─ Bordes / radios
```

└ Sombras
└ Animaciones

Y el logo tendrá simplemente su espacio reservado:

[LOGO] Sobre mí Proyectos Certificados Blog Contacto

Más adelante diseñamos el **I + π + Q**, y hacemos que ese logo se convierta también en una referencia de la identidad visual.

Los colores tampoco tienen que quedar grabados en piedra. Podemos empezar con una paleta y, si al construir el Hero vemos que no funciona, cambiarla desde el sistema de diseño sin tener que rehacer toda la web.

Internacionalización desde el principio

También me parece muy buena decisión empezar **ya utilizando recursos**, aunque solamente exista español.

Es decir, en lugar de tener textos repartidos directamente por los componentes:

```
<h1>Sobre mí</h1>
```

prepararemos desde el inicio el sistema para trabajar conceptualmente como:

```
<h1>@Localizer["About"]</h1>
```

y de momento:

```
About = "Sobre mí"  
Projects = "Proyectos"  
Certifications = "Certificaciones"  
Blog = "Blog"  
Contact = "Contacto"
```

Cuando añadamos inglés, añadiremos los recursos correspondientes, **sin duplicar las páginas ni los componentes**.

Esto es mejor que empezar en español "a pelo" y tener que refactorizar toda la aplicación después.

Y entonces sí: podemos empezar 🚀

Yo plantearía ahora el arranque exactamente así:

Commit 1

```
chore: initialize portfolio solution
```

Solución .NET 10, proyectos, referencias, configuración básica y Home temporal:

```
Portfolio
Aplicación en construcción
```

La aplicación debe **compilar y ejecutarse correctamente**, pero todavía sin diseño real.

Commit 2

```
feat: configure frontend design system
```

Tailwind + estructura CSS + tipografía + variables de diseño + temas + preparación de recursos de idioma.

Commit 3

```
feat: create portfolio layout and navigation
```

Navbar, Footer, layout general y navegación.

Commit 4

```
feat: create home hero section
```

Aquí empezaremos ya con el diseño que hemos imaginado:

foto + presentación corta + CTA + elemento tecnológico animado.

Y a partir de ahí iremos construyendo el Home sección por sección.

Y una cosa importante: **no necesitamos tener ahora tu foto ni el logo definitivo para hacer el primer commit ni para preparar la estructura del proyecto.** Cuando lleguemos al Hero y queramos afinar el diseño visual, ahí sí será útil que me pases la fotografía que quieras utilizar.

Por tanto, yo daría por **cerrada la fase de decisiones iniciales** y pasaría ya a crear la solución base del proyecto.

ok si ya lo tenemos todo listo, puedes generarme un markdown con un resumen de todo lo hablado y definido para el proyecto? con la estructura del proyecto que vamos a realizar, la arquitectura, lenguajes y tecnologías a usar, el flujo de trabajo a seguir y pasos a realizar como hemos ido definiendo, con explicaciones de por que hemos elegido cada uno de ellos? para incorporar este markdown a la raíz del proyecto y que tu como agente pueda leer este y saber como abordar el proyecto y comenzar a crearlo como hemos quedado finalmente

Sí. He preparado el Markdown como **documento de referencia principal del proyecto**, pensado precisamente para colocarlo en la raíz del repositorio y que pueda servir como contexto para futuros agentes/desarrolladores.

Incluye:

- Objetivo del portfolio.
- Stack completo y justificación de cada tecnología.
- Arquitectura por capas.
- Estructura de la solución.
- Blazor + Tailwind + MudBlazor.
- EF Core + Code First + migraciones.
- Identity y funcionamiento de `/admin`.
- Sistema de temas Sistema/Claro/Oscuro + `localStorage`.
- Internacionalización desde el inicio mediante recursos.
- Concepto del logo **I + π + Q**.
- Diseño previsto del Home.
- Hero, experiencia, proyectos, certificaciones, blog y contacto.
- Almacenamiento QNAP.
- Docker y despliegue.
- Flujo de trabajo iterativo.
- Estrategia de Git y commits.
- Orden completo de implementación.
- Principios que debemos respetar para evitar sobreingeniería.
- Estado inicial y próximo paso.

Puedes incorporarlo directamente a la raíz como:

`PORTFOLIO_PROJECT.md`

A partir de ahora, **este documento puede actuar como nuestro "contrato técnico" del proyecto**: cuando empecemos a crear la solución, seguiremos lo definido aquí y, si

durante el desarrollo tomamos alguna decisión importante que modifique la arquitectura o el flujo, actualizaremos también este Markdown para que permanezca sincronizado con el estado real del proyecto. ☐

Aunque ya hemos dejado cerrado el contrato del proyecto, me surge una duda para comentarte, si en vez de blazor para el frontend usara angular, si fuera viable y mejor o si siguiera siendo mejor opcion blazor

Sí, **Angular sería totalmente viable**, y de hecho la comparación con Blazor es bastante interesante para tu caso porque las dos opciones pueden producir un portfolio profesional. Pero teniendo en cuenta **qué quieres demostrar con este proyecto y tu perfil**, yo mantendría **Blazor** como primera opción.

La clave es que no estamos preguntando simplemente "¿qué frontend es mejor?", sino:

¿Qué frontend aporta más valor a ESTE portfolio?

Blazor vs Angular en tu proyecto

	Blazor	Angular
Backend	.NET 10	.NET 10
Lenguaje frontend	C#	TypeScript
Arquitectura	Componentes Blazor	Componentes Angular
SPA	Opcional	Orientado a SPA
SSR/SEO	Muy bueno con Blazor Web App	Muy bueno con Angular SSR
Integración con .NET	★★★★★	★★★★★
Ecosistema frontend	★★★★★	★★★★★
Mercado laboral frontend	★★★★	★★★★★
Curva inicial	Más sencilla para ti	Mayor
Demostración de C#/.NET	★★★★★	★★★★★

	Blazor	Angular
Demostración de TypeScript	—	★★★★★
Encaja con tu portfolio	★★★★★	★★★★★

Angular tiene una ventaja importante

Angular te permitiría demostrar una tecnología frontend profesional adicional.

Tendrías:

```
Angular
  ↓
TypeScript
  ↓
HTML / CSS
  ↓
ASP.NET Core API
  ↓
EF Core
  ↓
SQL Server
```

Y esto puede ser muy interesante para alguien que busque un perfil **full-stack .NET + frontend moderno**.

Además, Angular es especialmente fuerte para aplicaciones grandes y estructuradas: routing, formularios, DI, servicios, guards, HTTP client, testing, etc.

Pero hay un problema en TU caso

Tú ya me habías comentado que quieres hacer **otro proyecto específicamente con React** para demostrar React/TypeScript.

Por tanto, si hacemos:

```
Portfolio → Angular
Otro proyecto → React
```

estaríamos utilizando el portfolio principalmente para demostrar frontend.

Mientras que con:

Portfolio → Blazor + .NET
Otro proyecto → React + TypeScript

consequimos algo más interesante:

Proyecto 1 — Portfolio

Full-stack .NET

C#
.NET 10
ASP.NET Core
Blazor
EF Core
SQL Server
Identity
Docker

Proyecto 2 — Aplicación React

Frontend moderno independiente

React
TypeScript
ASP.NET Core API
SQL Server

Y de esa manera tu GitHub no muestra dos proyectos que utilizan prácticamente el mismo stack frontend.

Hay otra cuestión importante: nuestro diseño actual

La arquitectura que hemos planteado:

Portfolio.Web
Portfolio.Application
Portfolio.Domain
Portfolio.Infrastructure
Portfolio.Tests

encaja muy naturalmente con Blazor.

Con Angular tendríamos que cambiarla bastante:

```

Portfolio
|
├── frontend
|   └── Angular
|
└── backend
    ├── API
    ├── Application
    ├── Domain
    ├── Infrastructure
    └── Tests
  
```

Y aparecería una frontera bastante más clara:

```

Angular
  |
  | HTTP / JSON
  ▼
ASP.NET Core API
  |
  ▼
Application
  |
  ▼
Domain
  |
  ▼
Infrastructure
  |
  ▼
SQL Server
  
```

Esto **no es malo**. De hecho, arquitectónicamente puede ser incluso más interesante.

Pero para un portfolio personal añade bastante complejidad:

- API
- DTOs
- HTTP
- autenticación entre frontend/API
- CORS según configuración
- gestión de tokens/cookies
- dos proyectos frontend/backend
- dos ecosistemas de dependencias
- despliegue de frontend y backend

Mientras que con Blazor:

Blazor Web App



Application



Infrastructure



SQL Server

podemos mantener una solución bastante limpia.

¿Y para el SEO del portfolio?

Aquí tampoco descartaría Angular.

Angular moderno puede trabajar perfectamente con SSR y prerendering.

Pero **Blazor Web App** también nos permite plantear la parte pública con **renderizado del servidor**, que es precisamente lo que queremos para:

- Home
- Sobre mí
- Proyectos
- Certificaciones
- Blog

Por tanto, no elegiría Angular pensando que necesariamente tendrá mejor SEO.

Para este proyecto, **las dos opciones pueden hacerlo correctamente.**

Donde sí veo una diferencia importante

Para tu portfolio quiero que haya una cosa que se note:

"Este tío es desarrollador .NET y sabe construir una aplicación .NET moderna de principio a fin."

Y Blazor refuerza muchísimo ese mensaje.

Por ejemplo, puedes tener:

```
C#
↓
ASP.NET Core
↓
Blazor
↓
EF Core
↓
SQL Server
↓
Identity
↓
Docker
```

Todo dentro del mismo ecosistema.

Y además puedes decir en una entrevista:

"Construí mi portfolio como una aplicación Blazor Web App sobre .NET 10, utilizando EF Core Code First, Identity, SQL Server, Docker y un área administrativa."

Eso es bastante potente para una candidatura **.NET Developer**.

¿Significa esto que Angular sería peor?

No.

Si tu objetivo principal fuera:

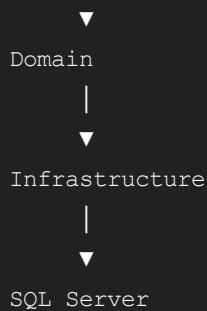
"Quiero utilizar este portfolio para demostrar que soy desarrollador frontend/full-stack y quiero potenciar Angular."

Entonces te diría:

Angular.

Incluso podríamos hacer una arquitectura muy buena:

```
Angular
|
▼
ASP.NET Core Web API
|
▼
Application
|
```



Y sería un proyecto técnicamente muy sólido.

Pero **no creo que sea la opción óptima para tu portfolio concreto.**

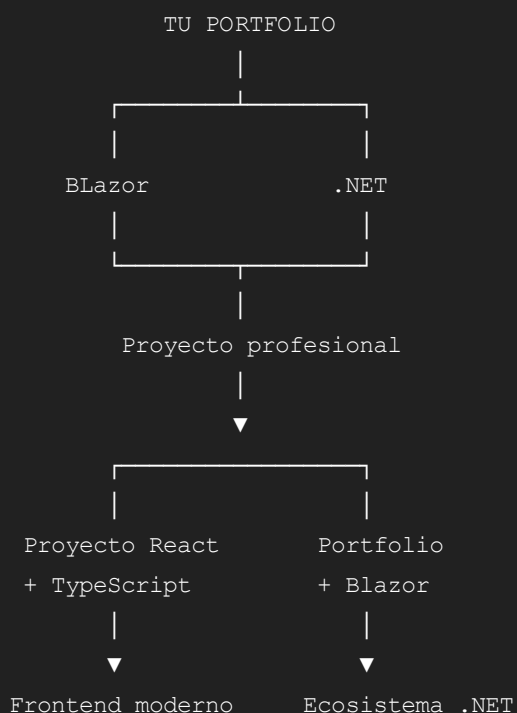
Mi recomendación final

Yo mantendría nuestra decisión:

● **Blazor Web App**

Y conservaría Angular como una tecnología que puedes demostrar posteriormente.

De hecho, veo una estrategia bastante buena para tu perfil:



Así puedes demostrar **dos perfiles complementarios**, en lugar de meter Angular, React y Blazor todos juntos en el mismo proyecto.

Y además, hay algo que considero especialmente importante: **no cambiaría ahora el contrato del proyecto solamente porque Angular sea una alternativa técnicamente**

válida. La arquitectura que hemos definido está bastante bien alineada con lo que quieres conseguir.

Por tanto: Angular es viable y profesional, pero para este proyecto concreto seguiría con Blazor.

vale sigo manteniendo lo que tenemos en el contrato con blazor. para ese segundo proyecto que hiciera con react + typescript, si quiero usar angular tambien ya seria elegir entonces entre react y angular? porque los son lo mismo y tendria que elegir entre una de ellas? si es asi pues haria un proyectito con react y otro con angular para tener esos dos conocimientos y presentarlos

Sí, exactamente. Y de hecho **me parece una estrategia bastante buena para tu CV**, porque así no intentas meter todas las tecnologías frontend en un único proyecto.

La idea quedaría muy coherente:

Proyecto 1 — Portfolio profesional

Blazor + .NET 10

```
Blazor Web App
  ↓
Application
  ↓
Domain
  ↓
Infrastructure
  ↓
SQL Server
```

Aquí demuestras principalmente:

- C#
- .NET / ASP.NET Core
- Blazor
- Entity Framework Core
- SQL Server
- Identity
- Docker
- Arquitectura por capas

- Diseño responsive
- Backend + frontend dentro del ecosistema .NET

Es tu proyecto principal y está muy alineado con el puesto de **.NET Developer** que estás buscando.

Proyecto 2 — React + TypeScript

Aquí sí tendría sentido hacer un proyecto específicamente pensado para demostrar:

- React
- TypeScript
- Componentes
- Hooks
- Gestión de estado
- Consumo de APIs REST
- Routing
- Formularios y validación
- Autenticación
- etc.

Y el backend podría ser nuevamente **ASP.NET Core Web API**, por ejemplo:

```
React + TypeScript
  ↓
  REST API
  ↓
ASP.NET Core
  ↓
  SQL Server
```

Así además demuestras que sabes separar frontend y backend mediante una API, que es diferente de la arquitectura del portfolio.

Proyecto 3 — Angular

Y aquí haría otro proyecto pequeño, pero **no un clon del proyecto React**.

Por ejemplo:

```
Angular + TypeScript
  ↓
  REST API
  ↓
```

ASP.NET Core



SQL Server

De esta forma puedes poner en tu CV:

Frontend: Blazor, React, Angular, TypeScript, JavaScript, HTML5, CSS3

Pero lo importante es que **cada tecnología está respaldada por un proyecto real** que puedes enseñar.

Y sí: React y Angular compiten en la misma zona

Tu planteamiento es correcto, aunque hay una pequeña diferencia conceptual:

	React	Angular
Tipo	Biblioteca/framework de UI	Framework completo
Lenguaje habitual	JavaScript / TypeScript	TypeScript
Arquitectura	Más flexible	Más estructurada
Routing	Librería habitual	Incluido
HTTP	Librería habitual	Incluido
Estado	Ecosistema de soluciones	Soluciones propias + ecosistema
Curva de aprendizaje	Generalmente más gradual	Generalmente más elevada
Mercado	Muy amplio	Muy amplio
Empresas grandes	Muy frecuente	Muy frecuente

Por eso **no necesitas elegir uno para siempre**.

Si tu objetivo fuera decir *"quiero aprender un único framework frontend además de Blazor"*, entonces sí te recomendaría elegir uno.

Pero en tu caso es diferente:

Quieres construir portfolio y demostrar amplitud tecnológica.

Entonces tiene bastante sentido aprender **los dos**, especialmente porque ya vas a tener Blazor.

De hecho, tu portfolio podría quedar muy potente

Tendrías tres proyectos que cuentan historias diferentes:

1. Portfolio

Blazor + .NET

"Sé construir una aplicación completa dentro del ecosistema .NET."

2. Aplicación React

React + TypeScript + ASP.NET Core API

"Sé trabajar con React y consumir un backend desacoplado."

3. Aplicación Angular

Angular + TypeScript + ASP.NET Core API

"También sé trabajar con Angular y su arquitectura más estructurada."

Y hay una ventaja adicional: **los proyectos 2 y 3 podrían compartir conceptos de backend**, pero no deberían ser exactamente la misma aplicación.

Por ejemplo:

- React → aplicación de gestión de entrenamientos/deporte
- Angular → aplicación de gestión de proyectos/tareas
- Blazor → tu portfolio profesional

Eso encaja además bastante bien con tu interés por **tecnología + deporte**.

Mi recomendación concreta

Yo mantendría **100 % la decisión actual de Blazor para el portfolio**.

Después:

1.º Blazor → ahora

2.º React + TypeScript → siguiente proyecto

3.º Angular + TypeScript → tercer proyecto

Y no intentaría hacer los tres a la vez. Primero terminaría el portfolio con una calidad alta. Después React y finalmente Angular.

Así, cuando un recruiter vea tu GitHub, no verá simplemente una lista de tecnologías que dices conocer, sino **tres proyectos que demuestran que realmente las has utilizado**.

podria realizar el proyecto con antigravity o es mejor visual studio para el tema de frameworks y demas, la usabilidad, etc

Sí, **podrías hacerlo con Antigravity**, pero para tu caso yo distinguiría entre "**puedo hacerlo**" y "**qué entorno me conviene más para este proyecto**".

Si por **Antigravity** te refieres al entorno de desarrollo de Google basado en agentes/IA, no hay una incompatibilidad especial con Blazor, .NET, React o Angular. Los frameworks no dependen de que uses Visual Studio.

Para tu portfolio Blazor

Yo usaría **Visual Studio** como entorno principal.

Tienes una combinación muy natural:

Visual Studio + .NET 10 + ASP.NET Core + Blazor + EF Core + SQL Server

Especialmente porque este proyecto es precisamente el que quieres presentar como tu proyecto principal de **.NET Developer**.

Visual Studio te da una experiencia muy integrada para:

- .NET / ASP.NET Core
- Blazor
- C#
- NuGet
- Entity Framework Core
- migraciones
- debugging
- Identity
- SQL Server
- Docker
- Git

- tests
- solución con varios proyectos

Y para tu arquitectura:

```
Portfolio.sln
|
├─ Portfolio.Web
├─ Portfolio.Application
├─ Portfolio.Domain
├─ Portfolio.Infrastructure
└─ Portfolio.Tests
```

Visual Studio encaja especialmente bien.

Para React + TypeScript

Aquí cambiaría bastante la recomendación.

Para el futuro proyecto React:

VS Code / Antigravity me parece más natural que Visual Studio.

```
React
TypeScript
Vite
ESLint
...
```

Y si utilizas Antigravity como asistente/agente de programación, puede ser especialmente interesante para ayudarte a generar componentes, refactorizar, crear tests, etc.

Y para Angular

También utilizaría **VS Code o Antigravity**:

```
Angular
TypeScript
Angular CLI
...
```

No necesitas Visual Studio para Angular.

Por tanto, yo haría esto

Proyecto	Tecnología	IDE que usaría
 Portfolio	.NET 10 + Blazor	Visual Studio
 Proyecto 2	React + TypeScript	Antigravity / VS Code
 Proyecto 3	Angular + TypeScript	Antigravity / VS Code

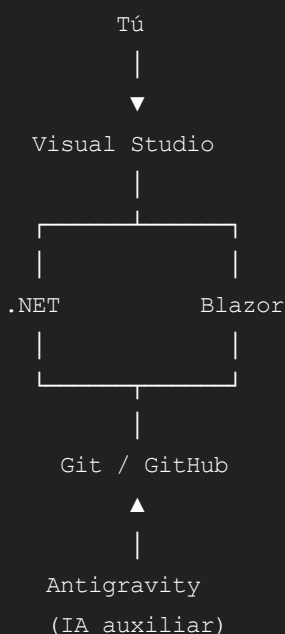
Y hay algo importante: **no tienes que utilizar el mismo IDE en los tres proyectos.**

De hecho, para ti puede ser positivo aprender a moverte por ambos entornos.

¿Y podemos usar Antigravity para ayudarnos con el portfolio?

Sí. Incluso podría tener sentido usarlo **como herramienta auxiliar**, mientras mantenemos Visual Studio como IDE principal.

Por ejemplo:



La clave sería que **la arquitectura, decisiones y código importante los controlemos nosotros**, y no dejar que el agente empiece a crear cosas arbitrariamente. Esto es especialmente importante en tu proyecto porque quieres que sea un proyecto que **puedas defender en una entrevista.**

De hecho, para este portfolio me parece mejor enfoque que dejar que una IA te genere toda la aplicación de golpe.

Visual Studio para construir y depurar + Antigravity para ayudarte a desarrollar, y vamos haciendo los commits pequeños que ya habíamos definido.

Así el proyecto sigue siendo realmente tuyo y además puedes explicar perfectamente por qué has elegido cada tecnología.

dime el prompt inicial para realizar la primera fase de creacion de la estructura del proyecto para darselo a copilot en este caso en visual studio

Sí. Para esta **primera fase** yo le daría a Copilot un prompt bastante controlado: que cree **solo la estructura de la solución**, referencias entre proyectos y una Home temporal funcional. No queremos todavía que empiece a crear entidades, EF Core, Identity, Tailwind, MudBlazor, etc.

Puedes copiarle directamente este prompt:

 Editar



Quiero iniciar desde cero un proyecto profesional de portfolio personal utilizando .NET 10 y Blazor Web App.

En esta primera fase NO quiero implementar todavía funcionalidades de negocio, base de datos, Entity Framework Core, Identity, Tailwind CSS, MudBlazor, TinyMCE ni el diseño visual definitivo. El objetivo es únicamente crear correctamente la estructura base de la solución para poder desarrollarla posteriormente por fases.

Objetivo de esta fase

Crear una solución llamada:

Portfolio.sln

Con los siguientes proyectos:

- Portfolio.Web
- Portfolio.Application
- Portfolio.Domain
- Portfolio.Infrastructure

- Portfolio.Tests

La arquitectura debe seguir una separación clara de responsabilidades y una dirección de dependencias coherente.

Arquitectura

La dependencia conceptual debe ser:

Portfolio.Web

↓

Portfolio.Application

↓

Portfolio.Domain

↑

Portfolio.Infrastructure

Es decir:

- Portfolio.Web puede depender de Portfolio.Application.
- Portfolio.Application puede depender de Portfolio.Domain.
- Portfolio.Infrastructure puede depender de Portfolio.Application y Portfolio.Domain.
- Portfolio.Domain NO debe depender de ningún otro proyecto de la solución.
- Portfolio.Application NO debe depender de Infrastructure ni de Web.
- Portfolio.Tests podrá referenciar los proyectos que sean necesarios para realizar pruebas posteriormente.

Evita crear patrones o abstracciones innecesarias. No quiero repositories, unit of work ni interfaces genéricas simplemente por seguir patrones. La arquitectura debe mantenerse sencilla y preparada para crecer.

Responsabilidad de cada proyecto

Portfolio.Domain

Debe contener únicamente elementos propios del dominio de la aplicación:

- Entities
- Value Objects si fueran necesarios

- Enums
- Domain Rules si fueran necesarias

Por ahora no crear entidades reales del portfolio porque todavía no hemos definido completamente el modelo de datos.

Crear únicamente la estructura de carpetas necesaria, sin inventar modelos de negocio.

No debe contener referencias a:

- Entity Framework
- ASP.NET Core
- Blazor
- SQL Server
- Infrastructure
- Web

Portfolio.Application

Será la capa de aplicación.

Debe quedar preparada para contener posteriormente:

- Application Services / Use Cases
- DTOs
- Validators
- Interfaces/abstracciones necesarias
- Lógica de aplicación

Por ahora no crear casos de uso ficticios ni servicios de ejemplo.

Puede depender de Portfolio.Domain.

Portfolio.Infrastructure

Será responsable posteriormente de:

- Entity Framework Core
- SQL Server
- DbContext
- EF Core Migrations
- ASP.NET Core Identity
- almacenamiento de archivos
- servicios externos

- email
- otras integraciones externas

En esta primera fase NO implementar todavía ninguno de ellos.

Puede depender de Portfolio.Application y Portfolio.Domain.

Portfolio.Web

Será la aplicación principal Blazor Web App.

Debe quedar preparada para contener:

- páginas públicas
- componentes Blazor
- layouts
- navegación
- administración
- autenticación/autorización posteriormente
- localización
- estilos

En esta fase solamente debe existir una Home temporal y funcional.

Portfolio.Tests

Proyecto destinado a pruebas automatizadas.

No es necesario crear todavía una batería de tests ficticios.

Únicamente dejar el proyecto correctamente configurado para poder añadirlos posteriormente.

Home temporal

Crear una página Home temporal muy sencilla que demuestre que la aplicación funciona correctamente.

Debe mostrar únicamente algo similar a:

Portfolio

Aplicación en construcción

No quiero todavía el diseño definitivo, Hero, navbar profesional, proyectos, certificados, blog, contacto, animaciones, Tailwind ni

estilos personalizados.

La finalidad es comprobar que:

1. La solución compila.
2. La aplicación Blazor arranca correctamente.
3. Las referencias entre proyectos son correctas.
4. La estructura está preparada para las siguientes fases.

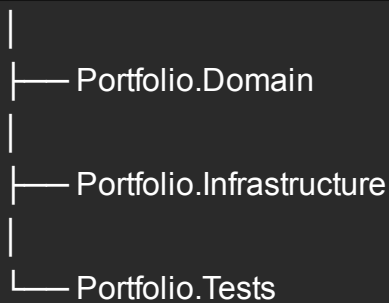
Reglas importantes

- Utilizar .NET 10.
- Utilizar Blazor Web App.
- Utilizar C#.
- Mantener nullable reference types habilitado.
- Mantener implicit usings habilitado cuando corresponda.
- Utilizar nombres de proyectos, namespaces y carpetas coherentes con los nombres anteriores.
- No instalar paquetes NuGet que todavía no sean necesarios.
- No configurar SQL Server todavía.
- No configurar EF Core todavía.
- No configurar Identity todavía.
- No configurar Tailwind todavía.
- No configurar MudBlazor todavía.
- No configurar TinyMCE todavía.
- No crear Docker todavía.
- No crear entidades de Projects, Certifications, BlogPosts, Experience, Contact, etc.
- No generar datos ficticios.
- No implementar funcionalidades que pertenezcan a fases posteriores.

Resultado esperado

Al finalizar esta fase quiero tener una solución limpia y compilable con esta estructura aproximada:

```
Portfolio.sln
|
|— Portfolio.Web
|
|— Portfolio.Application
```



Y las referencias de proyectos correctamente configuradas según la arquitectura indicada.

Antes de realizar cambios adicionales, revisa la solución completa y comprueba que:

- todas las referencias tienen sentido;
- no existen dependencias circulares;
- Domain permanece independiente;
- la aplicación Blazor funciona;
- la solución compila correctamente.

Si alguna decisión técnica necesaria para crear la estructura entra en conflicto con estas instrucciones, detente y explícame el conflicto antes de introducir una arquitectura diferente.

No avances a la siguiente fase. Esta tarea termina cuando la estructura base esté creada, compilando y funcionando.

Y una cosa importante para trabajar con Copilot

Yo no le daría todavía el `PORTFOLIO_PROJECT.md` entero y le diría "haz el proyecto". Es mejor este enfoque por fases.

Después de que Copilot termine esta primera fase, hacemos una revisión antes del siguiente prompt:

1. Estructura de solución
2. Referencias entre proyectos
3. `.csproj`
4. Program.cs
5. Home temporal
6. Compilación
7. Git

Y entonces haces el primer commit:

```
chore: initialize portfolio solution
```

Después pasamos a la **Fase 2: sistema visual + Tailwind + estructura base de la interfaz**, sin mezclar todavía base de datos ni administración.

Esto además nos permite controlar a Copilot para que **no sobreconstruya el proyecto desde el principio**, que es precisamente uno de los riesgos que quiero evitar en este proyecto.

una pregunta, para el prompt este inicial que me has dado, aunque me has comentado que no le de el portfolio_project.md completo, pero debería de adjuntárselo y decirle tras ponerle el prompt inicial que le adjunto el .md para que tenga en cuenta lo que se quiere hacer pero que solo se centre en la parte de la estructura primeramente como le indico en el prompt? o no le adjunto nada y con el prompt solo debe saber hacer lo que tenemos en el .md para la fase primera sin verlo? y otra cuestión, para el desarrollo del portfolio comentas que la estructura que va a tener es que va a estar compuesta de

Mostrar más

Sí, las dos preguntas son muy importantes antes de empezar, y haría una pequeña precisión respecto a lo que te dije antes.

1. ¿Le adjuntamos `PORTFOLIO_PROJECT.md` a Copilot?

Sí. En tu caso sí se lo adjuntaría.

Lo que quería evitar antes era darle el `.md` y decirle simplemente:

"Implementa todo lo que pone aquí."

Eso sí sería mala idea en esta fase.

Pero es perfectamente correcto darle el documento como **documentación de contexto del proyecto** y después delimitar claramente el trabajo que debe hacer ahora.

De hecho, es mejor porque el `.md` contiene decisiones que el prompt inicial por sí solo no reproduce completamente: arquitectura futura, diseño, administración, localización, Docker, workflow, etc.

La forma que te recomiendo es:

```
PORTFOLIO_PROJECT.md
    ↓
Documentación / contexto global
    ↓
Prompt de Fase 1
    ↓
Copilot implementa SOLO Fase 1
```

Y le dejaría muy claro:

"El documento describe el proyecto completo. No debes implementar ahora todo lo descrito en él. Úsalo únicamente como contexto y respeta las decisiones establecidas. En esta tarea concreta debes limitarte exclusivamente a la creación de la estructura inicial."

Incluso modificaría ligeramente el prompt

Al principio pondría algo como:

Contexto del proyecto:

Te adjunto `PORTFOLIO_PROJECT.md`, que contiene la especificación y decisiones globales del portfolio. Debes leerlo para comprender el objetivo, arquitectura, tecnologías y decisiones del proyecto.

IMPORTANTE: El documento describe el proyecto completo, pero en esta tarea NO debes implementar el proyecto completo. Esta es únicamente la **Fase 1: creación de la estructura base de la solución**. Debes implementar exclusivamente lo indicado en este prompt y no avanzar a fases posteriores.

Eso reduce muchísimo la posibilidad de que Copilot vea:

EF Core + Identity + Tailwind + MudBlazor + TinyMCE + Docker...

y empiece a instalar medio proyecto. 😊

Por tanto:

Sí → adjunta `PORTFOLIO_PROJECT.md`.

No → no le pidas que implemente el contenido completo del `.md`.

Y si el archivo está en la raíz del repositorio, todavía mejor:

```
Portfolio/
|
└─ PORTFOLIO_PROJECT.md
```

```
|  
└ ...
```

Así el propio repositorio tiene su documentación de proyecto desde el primer momento.

2. ¿Realmente necesitamos 5 proyectos?

Aquí hay más debate, y tu planteamiento **es totalmente válido**.

Podríamos hacer:

```
Portfolio.sln  
|  
└ Portfolio  
  |  
  ├── Domain  
  ├── Application  
  ├── Infrastructure  
  ├── Web  
  └ Tests
```

Es decir, **un único proyecto** y esas partes serían carpetas.

Y técnicamente funcionaría perfectamente.

Pero hay una diferencia fundamental:

Las carpetas NO crean fronteras arquitectónicas.

Por ejemplo:

```
Portfolio  
|  
└ Domain  
  |  
  └ Project.cs  
|  
└ Infrastructure  
  |  
  └ PortfolioDbContext.cs  
|  
└ Web  
  |  
  └ Home.razor
```

Aunque `Project.cs` esté dentro de `Domain`, desde C# puede hacer:

C#

```
using Microsoft.EntityFrameworkCore;
```

O:

C#

```
using Portfolio.Infrastructure;
```

No hay ninguna barrera real que se lo impida.

Las carpetas organizan.

Los proyectos `.csproj` crean límites de dependencia reales.

¿Entonces qué elegiría para TU portfolio?

En tu caso, mantendría los cinco proyectos.

No porque sea obligatorio para una aplicación Blazor, sino porque tu portfolio tiene una característica especial:

El propio código es parte de lo que quieres enseñar a un recruiter.

No es simplemente una web que quieres poner online.

Quieres que alguien pueda entrar en GitHub y ver:

```
Portfolio.sln
```

```
|— Portfolio.Web  
|— Portfolio.Application  
|— Portfolio.Domain  
|— Portfolio.Infrastructure  
|— Portfolio.Tests
```

y entender inmediatamente:

"Aquí hay una separación de responsabilidades."

Eso te interesa profesionalmente.

La diferencia se ve muy bien aquí

Opción A — un único proyecto

```
Portfolio
|
├─ Domain
├─ Application
├─ Infrastructure
├─ Web
└─ Tests
```

Más sencillo.

Menos proyectos.

Menos configuración.

Perfectamente válido para una aplicación pequeña.

Pero la arquitectura depende principalmente de **convenciones y disciplina**.

Opción B — varios proyectos

```
Portfolio.sln
|
├─ Portfolio.Web
|
├─ Portfolio.Application
|
├─ Portfolio.Domain
|
├─ Portfolio.Infrastructure
|
└─ Portfolio.Tests
```

Ahora tenemos restricciones reales:

```
Web
|
▼
Application
|
▼
Domain
```

e Infrastructure:

```
Infrastructure
|
```

```
└─ Application
└─ Domain
```

Mientras que:

```
Domain
```

no referencia absolutamente a nadie.

Si mañana alguien intenta hacer:

```
Portfolio.Domain
    ↓
Portfolio.Infrastructure
```

Visual Studio/.NET te lo va a poner mucho más difícil porque **esa referencia no existe**.

Eso es una ventaja arquitectónica real.

Y hay otra cosa que me gusta especialmente para tu proyecto

Piensa en lo que vas a poder explicar en una entrevista.

Si te preguntan:

"¿Por qué has separado Domain, Application e Infrastructure?"

Puedes responder algo como:

"Quería mantener el dominio independiente de la infraestructura. Las reglas y entidades del dominio no dependen de Entity Framework, SQL Server ni de Blazor. La capa de aplicación contiene los casos de uso y las abstracciones necesarias, mientras que Infrastructure implementa las dependencias externas."

Eso es bastante más demostrable si realmente tienes:

```
Portfolio.Domain
Portfolio.Application
Portfolio.Infrastructure
```

que si simplemente tienes:

```
Portfolio
└─ Domain
```

```
└─ Application
└─ Infrastructure
```

Pero ojo: tampoco quiero que sobrediseñemos

Aquí está el matiz importante.

No quiero que terminemos haciendo una arquitectura de 15 proyectos porque "Clean Architecture dice que hay que hacerlo".

Los cinco que tenemos son suficientes:

```
Portfolio.Web
Portfolio.Application
Portfolio.Domain
Portfolio.Infrastructure
Portfolio.Tests
```

Y dentro de ellos sí podemos utilizar carpetas.

Por ejemplo:

```
Portfolio.Domain
|
└─ Entities
└─ ValueObjects
└─ Enums
└─ Rules
```

y:

```
Portfolio.Application
|
└─ DTOs
└─ Interfaces
└─ Services
└─ Validators
```

y:

```
Portfolio.Infrastructure
|
└─ Persistence
|   └─ DbContext
|   └─ Configurations
|   └─ Migrations
```

```
|
├ Identity
├ Storage
├ Email
└ ExternalServices
```

Es decir:

Proyectos = límites arquitectónicos.

Carpetas = organización interna.

Mi decisión definitiva para tu proyecto

Yo dejaría:

```
Portfolio.sln
|
├ Portfolio.Web
├ Portfolio.Application
├ Portfolio.Domain
├ Portfolio.Infrastructure
└ Portfolio.Tests
```

Y **no cambiaría esta decisión** a un único proyecto.

Porque tu objetivo no es únicamente conseguir que el portfolio funcione. También quieres que el repositorio sea una **demostración de tus conocimientos de desarrollo .NET**.

Y esta estructura te permite demostrarlo sin llegar a una arquitectura exageradamente compleja.

Por tanto, antes de darle el prompt a Copilot

Yo haría exactamente esto:

① Crear repositorio

```
Portfolio
```

② Añadir

```
PORTFOLIO_PROJECT.md
```

como documentación global.

③ Abrir el proyecto en Visual Studio + Copilot.

④ Darle el prompt de Fase 1

Y añadiría al principio:

He adjuntado `PORTFOLIO_PROJECT.md`. Léelo antes de realizar cambios. Es la especificación global del proyecto y debe utilizarse como contexto. Sin embargo, esta petición corresponde exclusivamente a la Fase 1 y no debes implementar ninguna funcionalidad perteneciente a fases posteriores.

⑤ Copilot crea solamente:

```
Portfolio.sln
|
├─ Portfolio.Web
├─ Portfolio.Application
├─ Portfolio.Domain
├─ Portfolio.Infrastructure
└─ Portfolio.Tests
```

más las referencias correspondientes y la Home temporal.

⑥ Comprobamos juntos lo que ha generado.

Y solo cuando veamos que la estructura es correcta, hacemos el primer commit:

```
chore: initialize portfolio solution
```

Después de eso ya podemos pasar a la siguiente fase.

Me parece bastante mejor empezar así que dejar que Copilot genere 50 archivos de golpe y después intentar averiguar qué ha hecho.

y el gif donde lo añado, en la raíz donde voy a poner el .md y la cual esa raíz va a contener posteriormente las soluciones o tiene que estar dentro de cada solución un gif para cada proyecto?

Para el GIF de **tecnologías del portfolio**, yo no lo pondría en la raíz del repositorio ni crearía uno para cada proyecto.

Lo organizaría dentro de `Portfolio.Web`, porque es un recurso visual que pertenece a la interfaz pública del portfolio.

La estructura sería aproximadamente:

```
Portfolio/
|
├─ PORTFOLIO_PROJECT.md
├─ Portfolio.sln
|
├─ Portfolio.Web/
|   ├─ Components/
|   ├─ wwwroot/
|       ├─ images/
|       ├─ gifs/
|           └─ technologies.gif
|       ├─ icons/
|           └─ ...
|       └─ ...
|
├─ Portfolio.Application/
├─ Portfolio.Domain/
├─ Portfolio.Infrastructure/
└─ Portfolio.Tests/
```

¿Por qué dentro de `wwwroot`?

Porque en Blazor los recursos estáticos que la aplicación debe servir al navegador — imágenes, GIF, iconos, fuentes, etc.— tienen sentido dentro de `wwwroot`.

Por ejemplo:

```
Portfolio.Web/wwwroot/gifs/technologies.gif
```

Y posteriormente desde un componente Blazor:

HTML

```

```

¿Y si posteriormente haces otros proyectos?

Ahí sí cada aplicación/proyecto tendrá sus propios recursos.

Por ejemplo, cuando hagas tu proyecto React:

```
PortfolioReact/  
├─ src/  
├─ public/  
├─ ...  
└─ ...
```

Y el proyecto Angular tendrá su propia estructura.

Pero no necesitas duplicar el GIF en `Portfolio.Web`, `Portfolio.Application`, `Portfolio.Domain`, **etc.** Solo pertenece a la aplicación que lo muestra.

Una pequeña recomendación

Como todavía estamos en la **Fase 1**, yo **no añadiría todavía el GIF** si ese GIF forma parte del diseño definitivo del Hero.

Primero:

```
Portfolio.Web  
↓  
Home temporal
```

Después, cuando hagamos la fase del sistema visual/Hero:

```
Portfolio.Web  
├─ wwwroot  
│   ├── images  
│   ├── gifs  
│   ├── icons  
│   └─ ...
```

Así el primer commit sigue siendo realmente:

```
chore: initialize portfolio solution
```

y los recursos visuales entrarán con el commit correspondiente al diseño.

En resumen: `PORTFOLIO_PROJECT.md` y `Portfolio.sln` → raíz; **GIF** → `Portfolio.Web/wwwroot/...`. No uno por cada proyecto de la solución.

disculpa, me referia a inicializar git

Ah, sí 😊. Si te referías a **inicializar Git**, entonces cambia la respuesta.

En ese caso, **sí tiene sentido inicializar Git en la raíz del repositorio**, es decir, en la carpeta que contiene la solución y el `PORTFOLIO_PROJECT.md`.

La estructura inicial sería:

```
Portfolio/
|
├─ .git/
├─ .gitignore
├─ PORTFOLIO_PROJECT.md
├─ Portfolio.sln
|
├─ Portfolio.Web/
├─ Portfolio.Application/
├─ Portfolio.Domain/
├─ Portfolio.Infrastructure/
└─ Portfolio.Tests/
```

¿Por qué en la raíz?

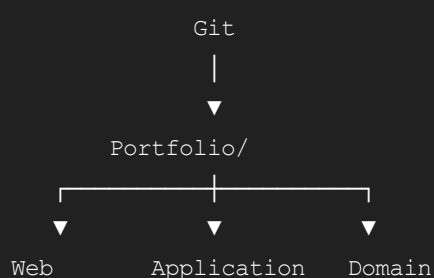
Porque **Git debe controlar todo el proyecto como una única unidad**.

No deberías hacer:

```
Portfolio/
├─ Portfolio.Web/
|   └─ .git/
├─ Portfolio.Application/
|   └─ .git/
├─ Portfolio.Domain/
|   └─ .git/
...
```

Eso convertiría cada proyecto en un repositorio Git independiente, que no es lo que queremos.

Queremos:



Todo forma parte del **mismo repositorio**.

Y el GIF que mencionabas

Si te referías a **Git**, entonces evidentemente el GIF no tiene nada que ver 😊.

La estructura correcta sería:

```
Portfolio/  
|  
├─ .git/  
├─ .gitignore  
├─ PORTFOLIO_PROJECT.md  
├─ Portfolio.sln  
|  
├─ Portfolio.Web/  
│   └─ wwwroot/  
|  
├─ Portfolio.Application/  
├─ Portfolio.Domain/  
├─ Portfolio.Infrastructure/  
└─ Portfolio.Tests/
```

Y `git init` se ejecutaría desde `Portfolio/`, la raíz.

En Visual Studio, si creas el repositorio desde el propio IDE, asegúrate igualmente de que la raíz del repositorio sea `Portfolio/`, no `Portfolio.Web/`.

Para tu primer commit, además, yo incluiría el `.md` **desde el principio**, porque es precisamente la documentación que explica qué es el proyecto y hacia dónde va.

o sea, que no haga git init dentro de la solución sino fuera de este, no? y otra cosa, la única solución que tendremos contiene todos los proyectos, no? porque en la estructura que me enseñas parece que los proyectos están al mismo nivel de la solución en vez de contener la solución a todos los proyectos

Sí, pero hay que distinguir **carpeta raíz del repositorio** de **carpeta de la solución**. Y tienes razón en la segunda observación: mi dibujo anterior podía dar a entender algo incorrecto.

1. ¿Dónde hacemos `git init`?

Si nuestra carpeta raíz del repositorio es `Portfolio/`, hacemos:

```
Portfolio/
```

y ahí:

Bash

```
git init
```

No hacemos `git init` dentro de `Portfolio.Web`, `Portfolio.Application`, etc.

Ahora bien, la solución (`Portfolio.sln`) también estará dentro de esa misma carpeta raíz.

Por tanto:

```
Portfolio/      ← raíz del repositorio Git
|
├─ .git/
├─ PORTFOLIO_PROJECT.md
├─ Portfolio.sln
|
├─ Portfolio.Web/
├─ Portfolio.Application/
├─ Portfolio.Domain/
├─ Portfolio.Infrastructure/
└─ Portfolio.Tests/
```

Es decir, **Git se inicializa en la carpeta que contiene la solución**, no en una carpeta "por encima" de la solución.

Mi frase anterior de "fuera de la solución" fue confusa. Lo correcto es:

Inicializamos Git en la raíz del repositorio, que será también la carpeta donde estará `Portfolio.sln`.

2. La solución contiene los proyectos

Exactamente.

Una solución de Visual Studio (`Portfolio.sln`) **no es una carpeta que físicamente contenga los proyectos**. Es un archivo que **agrupa y referencia** los diferentes

proyectos.

Por eso la estructura:

```
Portfolio/  
|  
├─ Portfolio.sln  
|  
├─ Portfolio.Web/  
├─ Portfolio.Application/  
├─ Portfolio.Domain/  
├─ Portfolio.Infrastructure/  
└─ Portfolio.Tests/
```

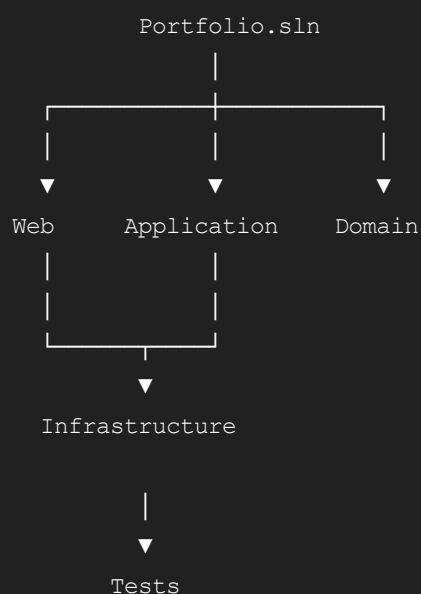
es totalmente correcta.

Visual Studio interpreta:

```
Portfolio.sln  
|  
├─ Portfolio.Web  
├─ Portfolio.Application  
├─ Portfolio.Domain  
├─ Portfolio.Infrastructure  
└─ Portfolio.Tests
```

Aunque físicamente sean carpetas hermanas.

Puedes imaginarlo así



La `.sln` es el **contenedor lógico**, no un directorio contenedor.

3. ¿Podríamos meterlos físicamente dentro de una carpeta

Portfolio?

También podríamos hacer:

```
Portfolio/  
|  
├─ PORTFOLIO_PROJECT.md  
├─ .git/  
|  
└─ src/  
    ├─ Portfolio.sln  
    ├─ Portfolio.Web/  
    ├─ Portfolio.Application/  
    ├─ Portfolio.Domain/  
    └─ Portfolio.Infrastructure/  
  
tests/  
└─ Portfolio.Tests/
```

Esto es una estructura bastante habitual en proyectos grandes.

Pero **para nuestro proyecto no creo que necesitemos complicarlo.**

Yo utilizaría:

```
Portfolio/  
|  
├─ .git/  
├─ .gitignore  
├─ PORTFOLIO_PROJECT.md  
├─ Portfolio.sln  
|  
├─ Portfolio.Web/  
├─ Portfolio.Application/  
├─ Portfolio.Domain/  
├─ Portfolio.Infrastructure/  
└─ Portfolio.Tests/
```

Es limpia, fácil de entender y suficientemente profesional.

Y esto conecta con lo que vamos a pedirle a Copilot

El flujo que yo haría ahora sería:

```
1. Crear carpeta Portfolio
```

↓


```
2. Inicializar Git en Portfolio/  
    ↓  
3. Añadir PORTFOLIO_PROJECT.md  
    ↓  
4. Crear Portfolio.sln  
    ↓  
5. Crear los 5 proyectos  
    ↓  
6. Añadir referencias entre ellos  
    ↓  
7. Crear Home temporal  
    ↓  
8. Compilar y comprobar  
    ↓  
9. Primer commit
```

Por tanto, **sí: tendremos una única solución** `Portfolio.sln` **y esa solución agrupará los cinco proyectos**. No tendremos una solución para `Web`, otra para `Domain`, otra para `Infrastructure`, etc.

Y el repositorio Git abarcará **todo el conjunto**.

ChatGPT puede cometer errores. Considera verificar la información importante.